

In [19]: *# code performing the needed calculations for the Theory.*

```
import cv2 as cv
import Img_moments as IM
import numpy as np

#image
img = np.zeros((5, 10), dtype=np.uint8)

# adds in the horizontal line
img[2, 2:7] = 1

# shows image
print("Image array:")
print(img)

# Calculate image moments
M = [IM.Raw_Img_Moment(img,0,0),
      IM.Raw_Img_Moment(img,1,0),
      IM.Raw_Img_Moment(img,0,1)]

# Display some of the moments
print("\nCalculated Moments:")
print(f"M_00: {M[0]}, M_10: {M[1]}, M_01: {M[2]}")

cx, cy = IM.Centroid(img)

I = IM.Hu_Moments(img, cx, cy)

print(f"I_1: {I[0]}, I_2: {I[1]}, \n I_3: {I[2]}, I_4: {I[3]}, \n I_5: {I[4]}, I_6: {I[5]}, \n I_7: {I[6]}")
```

Image array:

```
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 1 1 1 1 1 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]
```

Calculated Moments:

```
M_00: 5, M_10: 20, M_01: 10
I_1: 0.4, I_2: 0.16000000000000003,
I_3: 0.0, I_4: 0.0,
I_5: 0.0, I_6: 0.0,
I_7: 0.0
```

In [20]: *# the common libraries used in the previous code version.*

```
import pandas as pd
import os
from PIL import Image
import matplotlib.pyplot as plt

# custom libraries made to keep code clean and up to standard
import Helpers as H
import Score as S
import Binary as B
import Plotting as P

#ensures we are in the correct location
os.chdir(r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs")
```

In [21]: *# this is a set of paths that will be used later however should be defined separately for readability*

```
# this array stores all the images that work best with inverted theasholding
img_inv = [r"Data\Raw\Snowflake.jpeg",
            r"Data\Raw\snowflake_2.jpg",
            r"Raw\Spine.jpg",
            r"Raw\Bear.jpg",
            r"Raw\Test.jpg",
            r"Raw\Bone.jpg",
            r"Raw\Bone_2.jpg",
            #r'Raw\Triangle.jpg',
            r"Data\Shape\Raw\Bear_Shape.jpg",
            r"Data\Shape\Raw\Snow_shape.jpg",
            r"Data\Shape\Raw\Bone_Shape.jpg",
            r"Data\Shape\Raw\Square_shape.png",
            r'Data\Shape\Raw\spine_shape.jpg']

# no filling to be applied to these as it damages the quality of the images
no_fill = [r"Raw\Snowflake.jpeg",
            r"Raw\snowflake_2.jpg",
```

```

        r"data\Shape\Raw\Snow_shape.jpg"]

#this is the path of the folder which stores all the data folders
New_loc = r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data"

# this is the path for the shape folder path
shape_loc = r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape"

#this is the path of the code and all project related files, should be the current working directory at most ti
Old_loc = r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs"

# changes directory to the conv folder for the sake of saving the images
os.chdir(New_loc)

# a set of file paths for where code should be stored
raw_folder = os.path.join(New_loc, "Raw")
conv_folder = os.path.join(New_loc, "Conv")

Symm_folder = os.path.join(New_loc, "Symm")
Symm_folder_Hu = os.path.join(Symm_folder, "Hu")
Symm_folder_Raw = os.path.join(Symm_folder, "Raw")

# paths for the shape data which stores all shape related images
shape_raw_folder = os.path.join(shape_loc, "Raw")
shape_conv_folder = os.path.join(shape_loc, "Conv")

```

In [22]: # Here is the code used to convert the images into binary images and then save them in the Conv folder

```

count = 0

for img_interest in os.listdir(raw_folder):
    count += 1

    if img_interest.is_file():
        filename = os.path.basename(img_interest.path)

        file_path = rf"Raw\{filename}"

        if file_path in img_inv:
            Img_Bin = B.Img_Conv(file_path, inv = True)
            print(f"{filename} has undergone inverted binary thresholding")

        else:
            Img_Bin = B.Img_Conv(file_path, inv = False)
            print(f"{file_path} has undergone standard binary thresholding")

        # if there is an error it is likely due to the image not being binarised correctly
        cx, cy = IM.Centroid(Img_Bin)
        cx, cy = int(round(cx)), int(round(cy))
        img_conv = B.recenter_binary_image(Img_Bin, (cx, cy))

        img_conv = B.fill_black_spots(img_conv, file_path, no_fill)

        conv_save_path = os.path.join(conv_folder, filename)
        cv.imwrite(conv_save_path, img_conv)
        print(conv_save_path)

print(f"a total of {count} files have been converted into binary files.")

```

```

Bear.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\Bear.jpg
Bone.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\Bone.jpg
Bone_2.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\Bone_2.jpg
Raw\Snowflake.jpeg has undergone standard binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\Snowflake.jpeg
Raw\snowflake_2.jpg has undergone standard binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\snowflake_2.jpg
Spine.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\Spine.jpg
Raw\square.tif has undergone standard binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv\square.tif
a total of 7 files have been converted into binary files.

```

In [23]:

```

data = {}
#file_path = r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv"

print("Looping has started")
count = 0

```

```

for img_interest in os.listdir(conv_folder):

    filename = os.path.basename(img_interest.path)
    img_symm = cv.imread(img_interest.path, 2)

    #dimension values
    height, width = img_symm.shape
    #length = max(height, width) * 2

    #centroid calculation
    cx, cy = IM.Centroid(img_symm)
    cx, cy = int(round(cx)), int(round(cy))

    #if filename == "square.tif":
    #code for finding the degrees symmetry scores and moments of interest
    degrees, scores, moments_of_interest = S.find_symmetry_binary(img_symm.copy(), centroid=(cx,cy), height = h

    data[f"{filename}"] = [degrees, scores, moments_of_interest]
    print(filename)
    P.plot_degrees_vs_scores(degrees = degrees, scores = scores, filename = filename)

    count += 1
    print(f"{count} number of loops completed!")

```

Looping has started

Bear.jpg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bear_Plot.jpg'

1 number of loops completed!

Bone.jpg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bone_Plot.jpg'

2 number of loops completed!

Bone_2.jpg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bone_2_Plot.jpg'

3 number of loops completed!

Snowflake.jpeg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Snowflake_Plot.jpeg'

4 number of loops completed!

snowflake_2.jpg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\snowflake_2_Plot.jpg'

5 number of loops completed!

Spine.jpg

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Spine_Plot.jpg'

6 number of loops completed!

square.tif

Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\square_Plot.tif'

7 number of loops completed!

```

In [24]: symm_angles = [[0,57,129,178],
                        [101],
                        [0,180],
                        [0,34,70,78,158],
                        [13,154],
                        [40,166],
                        [125]]

symm_angles = [[180, 123, 51, 2],
               [79],
               [0,180],
               [180,146,110,102,22],
               [167,26],
               [140,14],
               degrees]

count = 0

for img_interest in os.listdir(conv_folder):

    if img_interest.is_file():
        filename = os.path.basename(img_interest.path)
        img = cv.imread(img_interest.path, 2)

        Cent = IM.Centroid(img)
        #cv.circle(img, data[filename][3], 5, (0,0,255), -1)

        if filename != "Test.jpg":
            img_symm = P.draw_all_symmetry_lines(img, centroid = Cent, angles=symm_angles[count])

        cv.imshow(filename, img_symm)
        cv.waitKey(0)
        cv.destroyAllWindows()
        cv.imwrite(fr"{Symm_folder_Raw}\\{filename}", img_symm)

```

```
count += 1
```

```
In [27]: # Bear, Bone, Bone_2, Snowflake, Snowflake_2, Spine, Square
symm_angles = [[180, 123, 51, 2],
               [79],
               [0, 180],
               [180, 146, 110, 102, 22],
               [167, 26],
               [140, 14],
               degrees]

#table = pd.DataFrame({"Bear": symm_angles[0],
#                      #
#                      "Bone": symm_angles[1],
#                      #
#                      "Bone_2": symm_angles[2],
#                      #
#                      "Snowflake": symm_angles[3],
#                      #
#                      "Snowflake_2": symm_angles[4],
#                      #
#                      "Spine": symm_angles[5],
#                      #
#                      "Square": symm_angles[6]})

names = ["Bear", "Bone", "Bone_2", "Snowflake", "Snowflake_2", "Spine", "Square"]
lines = [len(i) for i in symm_angles]
angles_interest = [sorted(i) for i in symm_angles]
angles_interest[-1] = ["0,1,2,...,180"]

expected_angles = [[180],[90,180],[90,180],[30,60,90,120,150,180],[30,60,90,120,150,180],[180],[45,90,135,180]]

symm_scores = [[round(data["Bear.jpg"][1][i],5) for i in symm_angles[0]],
               [round(data["Bone.jpg"][1][i],5) for i in symm_angles[1]],
               [round(data["Bone_2.jpg"][1][i],5) for i in symm_angles[2]],
               [round(data["Snowflake.jpeg"][1][i],5) for i in symm_angles[3]],
               [round(data["snowflake_2.jpg"][1][i],5) for i in symm_angles[4]],
               [round(data["Spine.jpg"][1][i],5) for i in symm_angles[5]],
               [round(data["square.tif"][1][i],5) for i in symm_angles[6]]]

table = pd.DataFrame({"Image name": ["Bear", "Bone", "Bone_2", "Snowflake", "Snowflake_2", "Spine", "Square"],
                      "Number of symmetry lines": lines,
                      "Angles expected": expected_angles,
                      "Angles of interest": angles_interest,
                      "Symmetry scores": symm_scores})

table
```

```
Out[27]:
```

	Image name	Number of symmetry lines	Angles expected	Angles of interest	Symmetry scores
0	Bear	4	[180]	[2, 51, 123, 180]	[0.00551, 0.0465, 0.03001, 0.00974]
1	Bone	1	[90, 180]	[79]	[0.0901]
2	Bone_2	2	[90, 180]	[0, 180]	[0.03845, 0.03845]
3	Snowflake	5	[30, 60, 90, 120, 150, 180]	[22, 102, 110, 146, 180]	[0.05617, 0.00472, 0.05701, 0.03096, 0.03255]
4	Snowflake_2	2	[30, 60, 90, 120, 150, 180]	[26, 167]	[0.00075, 0.01454]
5	Spine	2	[180]	[14, 140]	[0.09629, 0.00168]
6	Square	181	[45, 90, 135, 180]	[0,1,2,...,180]	[0.0, 4e-05, 9e-05, 0.00013, 0.00019, 0.00024,...]

```
In [28]: data = {}
keep = []
#file_path = r"C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Conv"

print("Looping has started")
count = 0
for img_interest in os.listdir(conv_folder):

    filename = os.path.basename(img_interest.path)
    img_symm = cv.imread(img_interest.path, 2)

    #dimension values
    height, width = img_symm.shape
    #length = max(height, width) * 2

    #centroid calculation
    cx, cy = IM.Centroid(img_symm)
    cx, cy = int(round(cx)), int(round(cy))

    #code for finding the degrees symmetry scores and moments of interest
    if filename == "square.tif":
```

```

degrees, scores, moments_of_interest = S.find_symmetry_binary(img_symm.copy(), centroid = (cx,cy), height = I
# here we want the code to only take the specific scores which are useful, for this we will only look at 1
score_av = np.array([[x[0] for x in scores], [x[1] for x in scores], [x[2] for x in scores], [x[3] for x in

data[f"{filename}"] = [degrees,
                        score_av,
                        S.find_local_minima(score_av),
                        (cy,cx),
                        moments_of_interest]

P.plot_degrees_vs_scores(degrees = degrees, scores = score_av, filename = filename, Hu = True)

degrees_of_interest = []
scores_of_interest = []

for i in data[f"{filename}"][2]:
    degrees_of_interest.append(degrees[i])
    scores_of_interest.append(score_av[i])

keep.append([degrees_of_interest, scores_of_interest, filename])

count += 1
print(f"{count} number of loops completed!")

```

Looping has started
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bear_Hu_Plot.jpg'
1 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bone_Hu_Plot.jpg'
2 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Bone_2_Hu_Plot.jpg'
3 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Snowflake_Hu_Plot.jpeg'
4 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\snowflake_2_Hu_Plot.jpg'
5 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\Spine_Hu_Plot.jpg'
6 number of loops completed!
Plot saved as 'C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Plots\square_Hu_Plot.tif'
7 number of loops completed!

```

In [29]: symm_angles = [[0,65,125],
                        [0,77,97],
                        [0,100,87],
                        [1,60,96,118,140],
                        [31,76,149,181],
                        [9,93],
                        [0,45,90,135]]

symm_angles = [[180,115,55],
                [180,103,83],
                [180,80,93],
                [179,120,84,62,40],
                [140,104,31,0],
                [171,87],
                [180,135,90,45]]

count = 0

for img_interest in os.listdir(conv_folder):
    if img_interest.is_file():
        filename = os.path.basename(img_interest.path)
        img = cv.imread(img_interest.path, 2)

        Cent = IM.Centroid(img)
        #cv.circle(img, data[filename][3], 5, (0,0,255), -1)

        if filename != "Test.jpg":
            img_symm = P.draw_all_symmetry_lines(img, centroid = Cent, angles = symm_angles[count])

            cv.imshow(filename, img_symm)
            cv.waitKey(0)
            cv.destroyAllWindows()
            cv.imwrite(fr"{Symm_folder_Hu}\\{filename}", img_symm)

            count += 1

```

```

In [30]: # Bear, Bone, Bone_2, Snowflake, Snowflake_2, Spine, Square

names = ["Bear", "Bone", "Bone_2", "Snowflake", "Snowflake_2", "Spine", "Square"]
lines = [len(i) for i in symm_angles]
angles_interest = [sorted(i) for i in symm_angles]

```

```

symm_scores = [[round(data["Bear.jpg"][1][i],5) for i in symm_angles[0]],
               [round(data["Bone.jpg"][1][i],5) for i in symm_angles[1]],
               [round(data["Bone_2.jpg"][1][i],5) for i in symm_angles[2]],
               [round(data["Snowflake.jpeg"][1][i],5) for i in symm_angles[3]],
               [round(data["snowflake_2.jpg"][1][i],5) for i in symm_angles[4]],
               [round(data["Spine.jpg"][1][i],5) for i in symm_angles[5]],
               [round(data["square.tif"][1][i],5) for i in symm_angles[6]]]

table = pd.DataFrame({"Image name": ["Bear", "Bone", "Bone_2", "Snowflake", "Snowflake_2", "Spine", "Square"],
                     "Number of symmetry lines": lines,
                     "Angles expected": expected_angles,
                     "Angles found": angles_interest,
                     "Symmetry scores": symm_scores})

table

```

Out[30]:

	Image name	Number of symmetry lines	Angles expected	Angles found	Symmetry scores
0	Bear	3	[180]	[55, 115, 180]	[0.00443, 0.02054, 0.01673]
1	Bone	3	[90, 180]	[83, 103, 180]	[0.00489, 0.00921, 0.04135]
2	Bone_2	3	[90, 180]	[80, 93, 180]	[0.02709, 0.0769, 0.06503]
3	Snowflake	5	[30, 60, 90, 120, 150, 180]	[40, 62, 84, 120, 179]	[0.0133, 0.00716, 0.01775, 0.01259, 0.01425]
4	Snowflake_2	4	[30, 60, 90, 120, 150, 180]	[0, 31, 104, 140]	[0.01289, 0.01871, 0.00487, 0.01039]
5	Spine	2	[180]	[87, 171]	[0.05158, 0.02697]
6	Square	4	[45, 90, 135, 180]	[45, 90, 135, 180]	[0.00039, 2e-05, 0.00039, 0.0]

In [31]: # A data frame which will store all data related to feature

```

feature_data = {
    "Image name": [],
    "Shape type": [],

    #"Centroid x position": [],
    #"Centroid y position": [],

    "Skewness in x": [],
    "Skewness in y": [],

    "Spread": [],
    "Orientation": [],
    "Eccentricity": []
}

Hu_data = {
    "Image name": [],
    "I_1": [],
    "I_2": [],
    "I_3": [],
    "I_4": [],
    "I_5": [],
    "I_6": [],
    "I_7": []
}

#creates the dataframes
df = pd.DataFrame(feature_data)
df_Hu = pd.DataFrame(Hu_data)

```

In [32]: #code that converts the images in the raw shapes folder into binary then stores them in the conv folder

```

os.chdir(Old_loc)
print("Looping has started")
count = 0
for img_interest in os.scandir(shape_raw_folder):

    filename = os.path.basename(img_interest.path)
    img = cv.imread(img_interest.path, 2)

    file_path = rf"Data\Shape\Raw\{filename}"

    if file_path in img_inv:
        Img_Bin = B.Img_Conv(file_path, inv = True)
        print(f"{filename} has undergone inverted binary thresholding")

```

```

else:
    Img_Bin = B.Img_Conv(file_path, inv = False)
    print(f"{file_path} has undergone standard binary thresholding")

#centroid calculation and basic conversion
cx, cy = IM.Centroid(img)
cx, cy = int(round(cx)), int(round(cy))
img_conv = B.recenter_binary_image(Img_Bin, (cx, cy))
img_conv = B.fill_black_spots(img_conv, file_path, no_fill)

conv_save_path = os.path.join(shape_conv_folder, filename)

cv.imwrite(conv_save_path, img_conv)
print(conv_save_path)

count += 1
print(f"{count} number of loops completed!")

```

Looping has started

```

Bear_Shape.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape\Conv\Bear_Shape.jpg
Bone_Shape.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape\Conv\Bone_Shape.jpg
Snow_shape.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape\Conv\Snow_shape.jpg
spine_shape.jpg has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape\Conv\spine_shape.jpg
Square_shape.png has undergone inverted binary thresholding
C:\Users\alloh\OneDrive\Desktop\coding\workspace\Term One Labs\Data\Shape\Conv\Square_shape.png
5 number of loops completed!

```

In [33]: *# Code that works to calculate the features for the set of test shapes*

```

count = 0
shape_data = []

for img_interest in os.listdir(shape_conv_folder):

    filename = os.path.basename(img_interest.path)
    image = cv.imread(img_interest.path, 2)

    #dimension values
    height, width = image.shape

    #centroid calculation
    cx, cy = IM.Centroid(image)
    cx, cy = int(round(cx)), int(round(cy))
    centroid = (cx, cy)

    #Central moments required for feature investigation
    Mu_00 = IM.Central_Order_Moments(img = image, p=0, q=0, x_cent=cx, y_cent=cy)

    Mu_11 = IM.Central_Order_Moments(img = image, p=1, q=1, x_cent=cx, y_cent=cy)
    Mu_02 = IM.Central_Order_Moments(img = image, p=0, q=2, x_cent=cx, y_cent=cy)
    Mu_20 = IM.Central_Order_Moments(img = image, p=2, q=0, x_cent=cx, y_cent=cy)

    Mu_30 = IM.Central_Order_Moments(img = image, p=3, q=0, x_cent=cx, y_cent=cy)
    Mu_03 = IM.Central_Order_Moments(img = image, p=0, q=3, x_cent=cx, y_cent=cy)

    # the calculations for the features of interest
    centroid = (cx, cy)
    theta = S.orientation(Mu_11=Mu_11, Mu_20=Mu_20, Mu_02=Mu_02)
    e = S.eccentricity(Mu_00=Mu_00, Mu_20=Mu_20, Mu_02=Mu_02)
    spread = S.Spread(Mu_00=Mu_00, Mu_20=Mu_20, Mu_02=Mu_02)
    skewness = S.Skewness(Mu_30 = Mu_30, Mu_03 = Mu_03, Mu_20 = Mu_20, Mu_02 = Mu_02)

    # Hu moments calculations as needed
    Hu = IM.Hu_Moments(img=image, x_cent=cx, y_cent=cy)

    shape_data.append({"Name": filename,
                      "Hu Moment": Hu,
                      "#Centroid": centroid,
                      "Spread": spread,
                      "Skewness": skewness,
                      "Orientation": theta,
                      "Eccentricity": e})

    count += 1
    print(f"{count} number of loops completed!")

```

1 number of loops completed!
2 number of loops completed!
3 number of loops completed!
4 number of loops completed!
5 number of loops completed!

```
In [34]: print("Looping has started")
count = 0

for img_interest in os.listdir(conv_folder):

    filename = os.path.basename(img_interest.path)
    image = cv.imread(img_interest.path, 2)

    # Get image dimensions
    height, width = image.shape

    # Compute centroid
    cx, cy = IM.Centroid(image)
    cx, cy = int(round(cx)), int(round(cy))

    # Central moments
    Mu_00 = IM.Central_Order_Moments(image, 0, 0, cx, cy)
    Mu_11 = IM.Central_Order_Moments(image, 1, 1, cx, cy)
    Mu_02 = IM.Central_Order_Moments(image, 0, 2, cx, cy)
    Mu_20 = IM.Central_Order_Moments(image, 2, 0, cx, cy)
    Mu_30 = IM.Central_Order_Moments(image, 3, 0, cx, cy)
    Mu_03 = IM.Central_Order_Moments(image, 0, 3, cx, cy)

    # Feature values
    centroid = (cx, cy)
    theta = S.orientation(Mu_11, Mu_20, Mu_02)
    e = S.eccentricity(Mu_00, Mu_20, Mu_02)
    spread = S.Spread(Mu_00, Mu_20, Mu_02)
    skewness = S.Skewness(Mu_30, Mu_03, Mu_20, Mu_02)

    # Hu moments
    Hu = IM.Hu_Moments(image, cx, cy)

    # Feature vector for comparison
    feature_vec = np.array([
        #cx, cy,
        spread,
        skewness[0], skewness[1],
        theta, e
    ])

    # Classification tracking variables
    group_curr_Hu = ""
    group_curr_feature = ""
    change = False
    best_diff = 1e-15
    give = 5
    offset = 3e-10

    # Compare to known shapes
    for group in shape_data:

        # Hu moments diff
        Hu_diff = np.sum(
            np.abs(group["Hu Moment"][:-2] - Hu[:-2]) / #we only take up to the 5th moment as last two are too
            (np.abs(Hu[:-2]) + np.abs(group["Hu Moment"][:-2]) + offset)
        )

        # Feature vector diff
        group_features = np.array([
            #group["Centroid"][0], group["Centroid"][1],
            group["Spread"],
            group["Skewness"][0], group["Skewness"][1],
            group["Orientation"], group["Eccentricity"]
        ])

        feature_diff = np.sum(
            (np.abs(group_features) - np.abs(feature_vec)) /
            (np.sum(np.abs(group_features)) + np.sum(np.abs(feature_vec)) + offset)
        )

        # HU match logic
        if Hu_diff < give:
            if change:
                if Hu_diff < best_diff:
                    group_curr_Hu = group["Name"]
                    best_diff = Hu_diff
```

```

    else:
        group_curr_Hu = group["Name"]
        best_diff = Hu_diff
        change = True

# Feature match logic
if feature_diff < give:
    if change:
        if feature_diff < best_diff:
            group_curr_feature = group["Name"]
            best_diff = feature_diff
    else:
        group_curr_feature = group["Name"]
        best_diff = feature_diff
        change = True

# Extract base filename (without extension)
name = filename.split(".")[0]

# Extract label up to underscore
final_shape_label = group_curr_feature.split("_")[0]

# Dataframes for normal features
new_df = pd.DataFrame({
    "Image name": name,
    "Shape type": final_shape_label,
    #"Centroid x position": [centroid[0]],
    #"Centroid y position": [centroid[1]],
    "Skewness in x": [skewness[0]],
    "Skewness in y": [skewness[1]],
    "Spread": spread,
    "Orientation": theta,
    "Eccentricity": e
})

# Extract label up to underscore
final_shape_label = group_curr_Hu.split("_")[0]

# Dataframes for Hu
new_Hu_df = pd.DataFrame({
    "Image name": name,
    "Shape type": final_shape_label,
    "I_1": [Hu[0]],
    "I_2": [Hu[1]],
    "I_3": [Hu[2]],
    "I_4": [Hu[3]],
    "I_5": [Hu[4]],
    "I_6": [Hu[5]],
    "I_7": [Hu[6]]
})

# Save results
df = pd.concat([df, new_df], ignore_index=True)
df_Hu = pd.concat([df_Hu, new_Hu_df], ignore_index=True)

count += 1
print(f"{count} number of loops completed!")

```

Looping has started

1 number of loops completed!
 2 number of loops completed!
 3 number of loops completed!
 4 number of loops completed!
 5 number of loops completed!
 6 number of loops completed!
 7 number of loops completed!

In [35]: display(df_Hu)

	Image name	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Shape type
0	Bear	0.000716	3.181192e-08	9.698744e-11	2.338827e-12	6.783561e-24	1.927309e-16	3.456596e-23	Bear
1	Bone	0.004526	1.956912e-05	4.984785e-11	2.410682e-11	8.267133e-22	1.012675e-13	1.220015e-22	Bear
2	Bone_2	0.016498	2.598999e-04	2.587316e-08	2.981509e-08	8.280552e-16	4.778230e-10	-7.879276e-18	Bear
3	Snowflake	0.001677	1.893518e-08	3.980096e-11	3.158929e-12	2.995033e-23	-3.907040e-16	1.891032e-23	Bear
4	snowflake_2	0.001924	1.191372e-08	2.559282e-11	3.044170e-11	4.282437e-22	-2.289472e-15	7.338848e-22	Bear
5	Spine	0.001327	4.546365e-07	3.575146e-10	1.629913e-10	2.176567e-20	9.987760e-14	3.277673e-20	Bear
6	square	0.000654	1.768214e-16	2.351268e-24	2.272205e-14	5.251970e-33	3.021446e-22	0.000000e+00	Square

```
In [36]: # First and last column from df_Hu
first_col = df_Hu.columns[0]
last_col = df_Hu.columns[-1]

# Middle columns: all columns in df that are not first_col or last_col
middle_cols = [col for col in df_Hu.columns if col not in [first_col, last_col]]

# Combine
new_order = [first_col, last_col] + middle_cols

# Reorder df
df_reordered = df_Hu[new_order]

df_reordered
```

Out[36]:

	Image name	Shape type	I_1	I_2	I_3	I_4	I_5	I_6	I_7
0	Bear	Bear	0.000716	3.181192e-08	9.698744e-11	2.338827e-12	6.783561e-24	1.927309e-16	3.456596e-23
1	Bone	Bear	0.004526	1.956912e-05	4.984785e-11	2.410682e-11	8.267133e-22	1.012675e-13	1.220015e-22
2	Bone_2	Bear	0.016498	2.598999e-04	2.587316e-08	2.981509e-08	8.280552e-16	4.778230e-10	-7.879276e-18
3	Snowflake	Bear	0.001677	1.893518e-08	3.980096e-11	3.158929e-12	2.995033e-23	-3.907040e-16	1.891032e-23
4	snowflake_2	Bear	0.001924	1.191372e-08	2.559282e-11	3.044170e-11	4.282437e-22	-2.289472e-15	7.338848e-22
5	Spine	Bear	0.001327	4.546365e-07	3.575146e-10	1.629913e-10	2.176567e-20	9.987760e-14	3.277673e-20
6	square	Square	0.000654	1.768214e-16	2.351268e-24	2.272205e-14	5.251970e-33	3.021446e-22	0.000000e+00

```
In [37]: display(df)
```

	Image name	Shape type	Skewness in x	Skewness in y	Spread	Orientation	Eccentricity
0	Bear		4.447009e-07	-0.000103	0.601510	0.004598	0.999845
1	Bone	Square	-1.679741e-05	0.000131	79.132186	0.034449	0.999902
2	Bone_2	Square	9.267455e-05	0.000188	76.399585	-0.039475	0.999960
3	Snowflake	Square	1.993008e-06	0.000090	1.179269	-0.009991	0.999752
4	snowflake_2	Square	-3.921397e-06	0.000065	1.011846	0.734904	0.999831
5	Spine	Square	-9.595002e-05	-0.000084	0.337100	-0.110129	0.999946
6	square	Bone	-2.206686e-06	-0.000002	1.000081	0.785398	0.999959